BUILT-IN OBJECTS: MATH, NUMBER, DATE

Scripting Programming Languages – Practical lesson 5

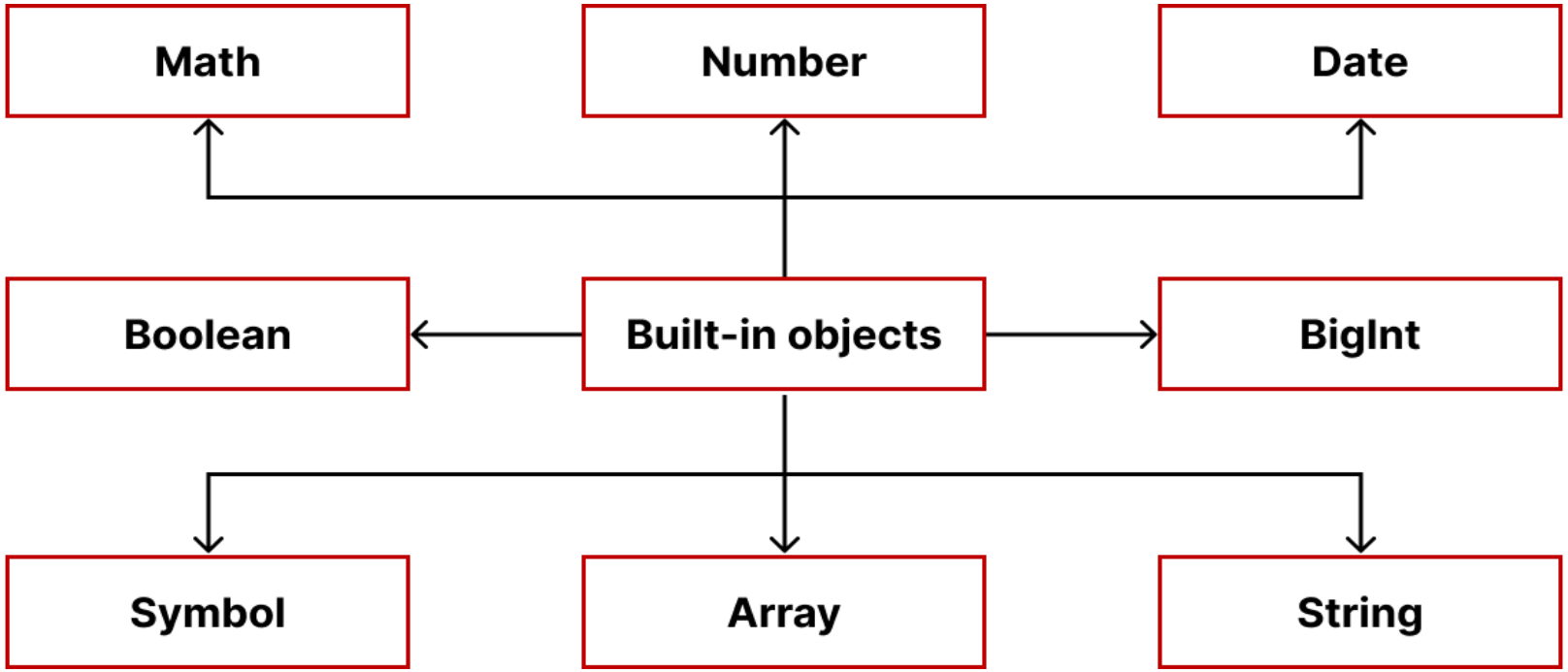
Julia Golubitskaya

TODAY

1. What built-in objects are
2. Wrapper objects
3. Math object and methods
4. Number object, methods and properties
5. Date object and methods
6. 4 tasks

WHAT BUILT-IN OBJECTS ARE

Built-in objects are property-objects of the global object with fixed names and their own properties.



WRAPPER OBJECTS

- For values of types **number, boolean, string** there are wrapper objects. They are needed when working with a value as an object (for example, if a method is called on the value).
- To get wrappers, the functions **Number(), Boolean(), String()** are used, called as constructors.

```
1  let str = "Hello";
2
3  let wrappedStr = new String("Hello");
4
5  console.log(typeof str); // "string"
6  console.log(typeof wrappedStr); // "object"
```

MATH OBJECT

The built-in **Math** object allows you to perform mathematical operations. Unlike other global objects, the **Math** object **is not a constructor**. Constants in JavaScript are defined with the full precision of real numbers:

Math.E	Constant e - Euler's number or the base of the natural logarithm, approximately equal to 2.718
Math.PI	Pi number, approximately equal to 3.14159
Math.SQRT2	Square root of two, approximately equal to 1.414
Math.SQRT1_2	Square root of 0.5, approximately equal to 0.707
Math.LN2	Natural logarithm of two. In JS rounded to 0.693147180559453
Math.LN10	Natural logarithm of 10, rounded to 2.302585092994046

MATH METHODS

Using the object's methods, you can perform various mathematical operations on numbers.

Math.log(x)	Returns the natural logarithm of a number
Math.floor(x)	Rounds a number down to the nearest integer
Math.round(x)	Rounds a number to the nearest integer
Math.ceil(x)	Rounds a number up to the nearest integer
Math.random()	Returns a random number between 0 and 1
Math.exp(x)	Returns E^x, where x is the argument and E is Euler's number (2.718...)
Math.sqrt(x)	Returns the square root of a number
Math.pow(x, y)	Returns the first number to the power of the second number
Math.max(x, y, …)	Returns the maximum value from a set of numbers
Math.min(x, y, …)	Returns the minimum value from a set of numbers

MATH EXAMPLE

The example demonstrates the use of the **Math** object method. All results are displayed on the console

```
1  // output integers in the range from -100 to 100
2  let number = Math.random() * 200 - 100;
3
4  document.writeIn(Math.round(number));
5
6
7  console.log(Math.pow(5, 2));
8  console.log(Math.sqrt(16));
9  console.log(Math.max(1, 5, 3, 9, 2));
10 console.log(Math.min(1, 5, 3, 9, 2));
```

NUMBER OBJECT

- In JavaScript, numbers can be of two types: integers and floating point.
- Numbers can be created in the usual way using variables and the assignment operator, without resorting to the **Number** object. However, this object has some useful properties and methods that can sometimes be useful.
- The **Number** object is a wrapper object that allows you to work with numeric values as objects.
- The **Number** object is created using the **Number()** constructor.
- In the context of a regular function (without the **new** operator), the **Number** object can be used for explicit type conversion.

```
const numObject = new Number(42);
console.log(numObject); // [Number: 42]
console.log(typeof numObject); // "object"
```

NUMBER METHODS

.toString()	Returns a string representation of the number in the numeral system with the specified base. If the parameter is not specified, the decimal system is assumed
.toFixed()	Represents the number in a form with a fixed number of digits after the decimal point, the count parameter is an integer that determines how many digits after the decimal point to specify.
.toPrecision()	Represents the number with a given total number of significant digits. The precision parameter is an integer that determines how many total digits, before and after the decimal point, to specify.
.parseInt()	The method parses a string argument and returns an integer.
.parseFloat()	The method parses a string argument and returns a floating point number.
.isNaN()	Checks if the value is NaN. Returns true or false respectively

NUMBER EXAMPLE

Example of creating an object:

```
1  let number1 = new Number(123);
2  let number2 = Number("unicorn");
3
4  console.log(number1); // 123
5  console.log(number2); // NaN
```

Example of number methods:

```
1  console.log(Number.parseInt("123.45")); // 123
2  console.log(Number.parseFloat("123.45Polina")); // 123.45
3  let number = 123.45678;
4  console.log(number.toPrecision(5)); // 123.45
```

NUMBER PROPERTIES

.MAX_VALUE	Represents the largest representable positive number
.MIN_VALUE	Represents the smallest representable positive number — that is, the positive number closest to zero
.NaN	Special value meaning "Not-a-Number".

```
1  function multiply(x, y) {
2      if (x * y > Number.MAX_VALUE) {
3          return ("Accepted as infinity");
4      }
5      return (x * y);
6  }
7
8  console.log(multiply(1.7976931348623157e+308, 2)); // Accepted
9  as infinity
```

DATE OBJECT

- The **Date** object allows you to perform various operations with date and time.

Using constructor:

```
// Using constructor:
date = new Date(); // date - current date
console.log(date); // Sat Dec 17 2022 00:15:39 GMT+0300 (Moscow Standard Time)
```

With string parameters:

```
// if time isn't specified, it's set to midnight GMT according to the time zone
let date = new Date("2017-01-26");
console.log(date); // Thu Jan 26 2017 03:00:00 GMT+0300 (Moscow Standard Time)

// new Date(year, month, date, hours, minutes, seconds, ms).
// Only the first two arguments are required.
let date = new Date("2021", "12");
console.log(date); // Sat Jan 01 2022 00:00:00 GMT+0300 (Moscow Standard Time)
```

DATE METHODS

.getDate()	Returns the day of the month (can be from 1 to 31) of the given date
.getDay()	Returns the day of the week (can be from 0-6, where 0 is Sunday and 6 is Saturday) of the given date
.getFullYear()	Returns the year (4 digits, e.g. 2017) of the given date
.getHours()	Returns the hour (can be from 0-23) of the given date
.getMinutes()	Returns the minutes (can be from 0-59) of the given date
.getMonth()	Returns the month (can be from 0-11) of the given date
.getSeconds()	Returns the seconds (can be from 0-59) of the given date
.toDateString()	Converts the date part of the object to a string
.toString()	Converts the object to a string
.toTimeString()	Converts the time part of the object to a string

DATE EXAMPLE

- It is important to note that in Date object methods, numbering starts from zero.

```
1  month = date.getMonth() + 1; // output current month
2
3  const now = new Date();
4  console.log(now.toString()); // contains day of week, date,
5  time, and timezone
6  console.log(now.toDateString()); // day of week, month, day,
7  and year only
8  console.log(now.toTimeString()); // time and timezone only
```

TASK 1

Random number and rounding

- Using **Math.random()**, generate a random floating-point number from 0 to 50.
- Round it to the nearest integer with **Math.round()** and log it.
- Round the same number up (**Math.ceil()**) and down (**Math.floor()**).
- Output all three results in one line separated by commas.

Example of console results:

```
Original number: 26.654546153405832
Math.round(), nearest integer:   27
Math.ceil(), rounded up:         27
Math.floor(), rounded down:       26
```

TASK 2

Number formatting

- Take the number 9.87654.
- Using ***Number*** object methods, convert it to a string with two decimal places (**.toFixed(2)**), to exponential notation with one decimal place (**.toExponential(1)**), and to a string with three significant digits (**.toPrecision(3)**).
- Output each result to the console on a separate line.

Example of console results:

```
toFixed(2):       9.88
toExponential(1): 9.9e+0
toPrecision(3):   9.88
```

TASK 3

Today’s date

- Create a `today` object with the current date and time.
- Output to the console: the day of the week (**.getDay()**), the month number (**.getMonth()** + 1), and the year (**.getFullYear()**).
- Briefly explain what the resulting day-of-week number means (e.g., "0 – Sunday").

Example of console results:

```
Day of week (getDay()):       4
Month number (getMonth() + 1): 6
Year (getFullYear()):          2026

 Day of week meaning:
   4 - Thursday

 Full date: Thu Jun 18 2026
```

TASK 4

Calculating age

- Create a `birthday` object with your birth date (e.g., `**new Date(2000, 0, 1)**` for January 1, 2000).
- Calculate your age in years by subtracting the birth year from the current year, and check whether your birthday has already occurred this year. If your birthday has not yet occurred, reduce the age by 1.
- Output the phrase: "*I am <age> years old*". Use the methods **.getFullYear()**, **.getMonth()**, and **.getDate()**.

Example of console results:

```
Birthday: Sat Jan 01 2000
Today:    Thu Jun 18 2026

Age calculation:
   Current year - Birth year = 2026 - 2000 = 26
   Birthday has occurred this year
   Age adjustment: No change

 "I am 26 years old"
```

TEST QUESTIONS

1. What are wrapper objects in JavaScript?
2. Why are wrapper objects needed if we have primitive types (string, number, boolean)?
3. What does Math.random() do and how do I get a random number between 1 and 10?
4. What is the difference between the Math.round(), Math.floor(), and Math.ceil() methods?
5. What is the difference between the toDateString(), toTimeString(), and toString() methods of the Date object?

THANK YOU FOR ATTENTION

Scripting Programming Languages – Practical lesson 5

Julia Golubitskaya